

And now we're ready for the **Layer 1 debugging exercise**. I'll give you a route and code, and you tell me what happens. I'll deliberately mix **dynamic URL segments, cookies, searchParams ****, fetch ****, no-store ****, and revalidate** so you have to distinguish all of them.

Absolutely 🤔🔥 This is exactly the kind of exercise that will make the **Layer 1 vs Layer 2 mental model** stick.

🧩 Layer 1 Debugging Exercise

Let's use this route:

/products/[category]



app/products/[category]/page.tsx

```
import { cookies } from "next/headers";

export default async function ProductsPage({
  params,
  searchParams,
}: {
  params: Promise<{ category: string }>;
  searchParams: Promise<{ sort?: string }>;
}) {
  const { category } = await params;
  const { sort } = await searchParams;

  const cookieStore = await cookies();
  const userId = cookieStore.get("userId")?.value;

  const products = await fetch(
    `https://api.example.com/products/${category}?sort=${sort}`,
    {
      next: {
        revalidate: 60,
      },
    }
  ).then((res) => res.json());
```

More actions

```
const recommendations = await fetch(
  `https://api.example.com/recommendations/${userId}`,
  {
    cache: "no-store",
  }
).then((res) => res.json());

return (
  <div>
    <h1>{category}</h1>
    <p>Sort: {sort}</p>
    <p>User: {userId}</p>
    <p>Products: {products.length}</p>
    <p>Recommendations: {recommendations.length}</p>
  </div>
);
}
```

And imagine the browser requests:

/products/electronics?sort=price



with:

Cookie:
userId=123



More actions

Your job

Don't worry about the exact **Next.js version-specific cache API behavior** yet. We're focusing on our **Layer 1 mental model**:

Browser Request

↓

Next.js receives request

↓

Can this route be static or dynamic?

↓

What causes dynamic rendering?

↓

What happens to each fetch?

↓

What gets cached?



Answer these **one by one**:

1. `[category]` → does the dynamic URL segment itself make the route dynamic at request time?
2. `searchParams` → does `?sort=price` make the route dynamic?
3. `cookies()` → what does this do to the route's rendering decision?
4. The first `fetch()` has:

```
next: { revalidate: 60 }
```

Is **that fetch's data** cacheable?

5. The second `fetch()` has:

```
cache: "no-store"
```

What happens to **that fetch's data**?

6. Most importantly:
Even though the first fetch says `revalidate: 60`, does that automatically mean the **whole** `/products/[category]` **route** gets stored as a static HTML response?
7. Finally, classify each thing:

Thing	Your classification
<code>[category]</code>	?
<code>searchParams</code>	?
<code>cookies()</code>	?
Products <code>fetch</code>	?
Recommendations <code>fetch</code>	?
Whole route	?

More actions

One rule to keep in your head

We're deliberately separating:

"Does this make the route dynamic?"

from:

"Can this particular data be cached?"

Those are **not** the same question.

Your turn — answer the 7 questions, and I'll debug your reasoning rather than just giving you the answers. 🚀



- 1) not it itself
- 2)yes
- 3)dynamic redering
- 4) yes and ISR model
- 5)No and SSR model
- 6)No bcoz of cookies
- 7)dynamic route ,dynamic route,cookies->dynamic,products-cached,rec-no cache,whole-dynamic

Yes! 🔥 You've got the **core Layer 1 model** right. There are just **two important corrections/clarifications** that will make your mental model much stronger.

Let's debug each answer

1. [category] → Dynamic URL segment

Your answer: "not it itself" ✅

More actions

Correct.

/products/[category]



means:

/products/electronics
/products/books
/products/shoes



The [category] is simply a **dynamic route parameter**.

It does **not** automatically mean **SSR/dynamic rendering**.

For example, a route can potentially be statically generated for known categories using:

```
generateStaticParams()
```

So:

Dynamic segment $\neq$ Dynamic rendering



★ Important distinction.

2. `searchParams` → Dynamic rendering

Your answer: yes

For this exercise, yes.

```
/products/electronics?sort=price
```



`searchParams` is request-specific information.

So the rendered result can depend on:

```
?sort=price  
?sort=name  
?sort=rating
```



More actions

Therefore, using `searchParams` in the page makes the route **dynamic/request-dependent** in the Layer 1 model we're learning.

3. `cookies()` → Dynamic rendering

Your answer: dynamic rendering

Exactly.

```
const cookieStore = await cookies();
```

The server needs the **incoming request's** cookie.

For example:

```
Request A
Cookie: userId=123
  ↓
Recommendations for user 123
```



versus:

```
Request B
Cookie: userId=456
  ↓
Recommendations for user 456
```



Therefore Next.js can't treat the route as one universal static HTML result.

```
cookies()
  ↓
request-specific data
  ↓
dynamic rendering
```



4. Products fetch

```
fetch(url, {
  next: {
    revalidate: 60,
  },
})
```

More actions

Your answer: "yes and ISR model" ⚠️

First half: yes.

The **fetch's data is cacheable** with a 60-second revalidation period.

Conceptually:

```
Products API
  ↓
Data Cache
  ↓
60 seconds
```



But I'd change this part:

✗ "this fetch = ISR model"

to:

✓ "This fetch uses time-based data caching/revalidation."

Why?

Because **ISR** is a **route-level rendering/caching concept**, whereas:

```
next: { revalidate: 60 }
```

is telling Next.js about the **fetch/data**.

This distinction is SUPER important:

FETCH LEVEL

```
next: { revalidate: 60 }
```

↓

data can be cached/revalidated

ROUTE LEVEL

static route + Full Route Cache

↓

ISR-like behavior



More actions

So don't automatically translate:

```
revalidate: 60
```



into:

```
"This entire route is ISR"
```



5. Recommendations fetch

```
fetch(url, {  
  cache: "no-store",  
})
```

Your answer: "No and SSR model" ⚠️

Again, first half is right:

```
cache: "no-store"
      ↓
don't use persistent Data Cache
      ↓
fetch fresh data
```



But I'd avoid saying:

"no-store = SSR"

Instead:

"no-store means this particular fetch isn't cached."

The route may be dynamically rendered for **other reasons too**.

In our example, we already have:

```
searchParams
cookies()
```



making the route request-dependent.

So:

More actions

```
recommendations fetch
      ↓
no-store
      ↓
fresh request
```



Not:

```
no-store
      ↓
SSR
```



That's the subtle distinction.

6. Does `revalidate: 60` make the whole route static?

Your answer: No because of cookies ✅🔥

Exactly.

We have:

```
cookies()
```

and:

```
searchParams
```

So the page depends on the current request.

Therefore:

```
/products/electronics?sort=price
+
Cookie: userId=123
↓
request-specific rendering
↓
Dynamic route rendering
```



More actions

The fact that **one fetch is cacheable** doesn't magically turn the entire page static.

7. Final classification

Your answer:

Thing	Your answer	Verdict
[category]	dynamic route	🟡 Dynamic segment, but not necessarily dynamically rendered
searchParams	dynamic route	✅ Causes request-dependent/dynamic rendering
cookies()	dynamic	✅
Products fetch	cached	✅

Thing	Your answer	Verdict
Recommendations fetch	no cache	✓
Whole route	dynamic	✓

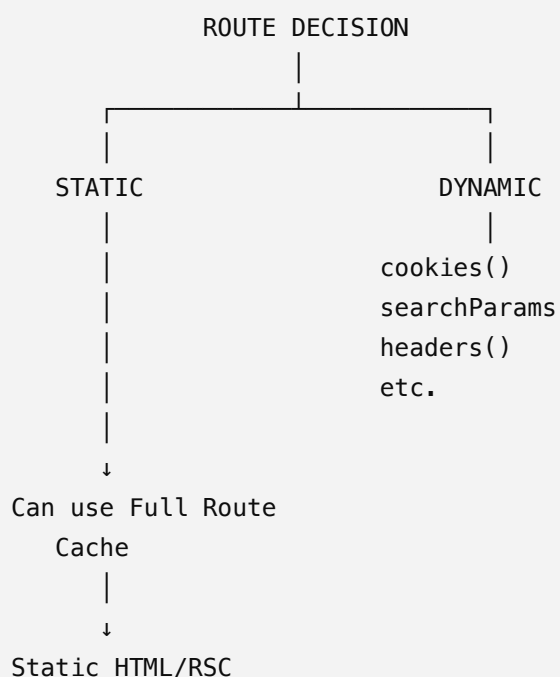
So I'd rewrite your table as:

Code	Layer 1 meaning
<code>[category]</code>	Dynamic route parameter
<code>searchParams</code>	Dynamic rendering trigger
<code>cookies()</code>	Dynamic rendering trigger
Products <code>fetch</code>	Cached data, revalidated every 60s
Recommendations <code>fetch</code>	Uncached/fresh data
Whole route	Dynamic rendering

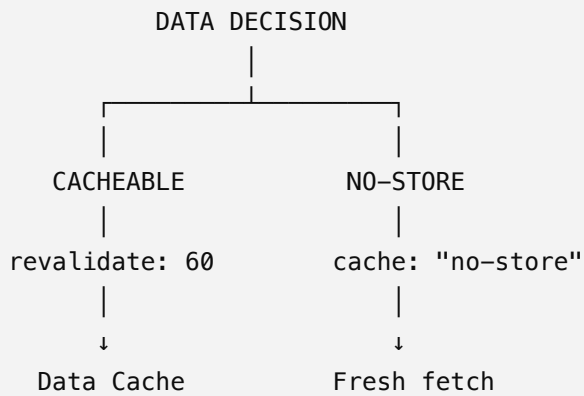
The BIG mental model

This is the piece I really want you to lock in:

More actions



Separately, inside the route:



And these two decisions can coexist:

WHOLE ROUTE
↓
Dynamic rendering

BUT

Products fetch
↓
Cached

AND

Recommendations fetch
↓
Not cached

🔥 That's exactly why we separate Layer 1 from Layer 2.

More actions

The route being dynamic **does not mean every piece of data inside it must be uncached.**

And a fetch being cached **does not mean the entire route becomes static.**

That's the distinction you were circling around in our earlier caching discussion.



ChatGPT can make mistakes. Check important info.